CET 3136C – Logic Devices Programming

Spring 2026

FPGA Square Wave Synthesizer

Final Project

Performed by

Abigail John

Josué Bouchard

Submitted to

Prof. Ashley Evans

CRN: 22475

Department of Electrical and Computer Engineering Technology (ECET)

Division of Engineering, Technology, and Advanced Manufacturing

Valencia College - West Campus

April 30, 2026

Contents

1. Introduction

Polyphonic music is a musical texture characterized by the simultaneous interweaving of multiple independent melodic lines, or voices. Performing it digitally requires not only computing each voice independently, but also transforming, scaling, and mixing them under strict timing constraints. As an example, playing 8 voices at CD-quality sample rate (44.1 kHz) requires calculating the next state for all 8 oscillators and mixing them every 22.7 microseconds. An FPGA is well-suited for this task because it allows each voice to be implemented as an independent hardware datapath running in parallel, eliminating the sequential bottleneck inherent to software execution.

The objective of this project is to create a synthesizer capable of producing square-wave voices, mixing them, and producing an I²S output signal sampled at around 45.95 kHz, capable of 8 simultaneous voices, using the DE10-Lite board containing the MAX-10 FPGA.

2. The Mathematics of Generating Sound

2.1. Generating a Square Wave of Arbitrary Frequency

If we have a clock of frequency f_{in}, a frequency divider is a circuit that, by using an internal counter from 0 to N, outputs a signal with f_{out}. The counter completes one full output period every N input clock cycles.

$$T_{\text{out}} = T_{\text{in}} \times N \iff f_{\text{out}} = \frac{f_{\text{in}}}{N} \tag{1}$$

Where T_{in} is the period of the input clock, the amount of time it takes for the input to complete a period; T_{out} is the output period, and N is the number to which the counter must count.

A corollary from Equation 1 is that, to obtain the N value for a counter, given f_{in} and a desired f_{out} we can obtain it by:

$$N = \left\lfloor \frac{f_{\text{in}}}{f_{\text{out}}} \right\rceil \tag{2}$$

A toggle divider is similar to a frequency divider, but it changes its output whenever the counter reaches N. This means that for the toggle divider:

$$T_{\text{toggle}} = 2 \times N \times T_{\text{in}} \iff f_{\text{toggle}} = \frac{f_{\text{in}}}{2 \times N} \iff N = \frac{f_{\text{in}}}{2 \times f_{\text{toggle}}} \tag{3}$$

2.2. The Equal Temperament Tuning

Among the most common methods of tuning, equal temperament divides an octave into 12 equal parts to obtain the frequency.

Starting with a base frequency f_0, we know that the frequency of the next octave, located 12 semitones higher, will be $f_{12} = 2 \times f_0$. To find the frequency for an arbitrary semitone f_i, we divide the octave into equal parts:

$$\begin{aligned}
&f_i = a^i \times f_0 \\
&f_{12} = 2 \times f_0 = a^{12} \times f_0 \\
&\\
&\implies 2 \times f_0 = a^{12} \times f_0 \\
&\implies 2 = a^{12} \\
&\implies a = \sqrt[12]{2} \\
&\implies \boxed{f_i = \left(\sqrt[12]{2}\right)^i \times f_0}
\end{aligned} \tag{4}$$

If we take $f_{\text{A4}} = 440\ \text{Hz}$ as a reference, we get the formula:

$$f_i = \left(\sqrt[12]{2}\right)^i \times 440\ \text{Hz} \tag{5}$$

where $i \in \mathbb{Z}$ is going to be the distance in semitones from A4, also known as concert A.

2.3. The I²S Format

I²S (Inter-IC Sound) is a serial protocol for transmitting digital audio between devices. It requires a zero-centered signal, meaning samples must be signed integers symmetric around zero.

A sample x is a signed integer of B bits representing the instantaneous value of the audio signal. It must be remembered that the range of the signed integer is:

$$x \in \left[-2^{(B-1)}, 2^{(B-1)} - 1\right] \tag{6}$$

For any type of signal, we can compute the peak amplitude (usually just called amplitude) as:

$$A(x) = \max_t |x(t)| \tag{7}$$

Because of Equation 6, the amplitude of x, $A(x)$ must always be:

$$A(x) \leq 2^{(B-1)} - 1 \tag{8}$$

The sample rate f_s is the number of audio samples transmitted per second. It sets a hard ceiling on the highest frequency that can be represented digitally:

$$f_{\max} = \frac{f_s}{2} \tag{9}$$

Equation 9 shows the Nyquist limit. For example, a sample rate of $f_s = 44\,100$ Hz can represent frequencies up to 22 050 Hz, which comfortably covers the full range of human hearing.

I²S uses two clocks derived from f_s:

- The bit clock (BCLK) runs at:

$$f_{\text{BCLK}} = f_s \times B \times C \tag{10}$$

  where:
  - B is the bit depth, the number of bits used to represent each sample.
  - C is the number of channels (1 for mono, 2 for stereo).

- The word select clock (LRCLK) runs at:

$$f_{\text{LRCLK}} = f_s \tag{11}$$

The BCLK and LRCLK signals can be obtained from the system clock f_{in} using the frequency divider, whose count can be derived from Equation 2:

$$\begin{aligned} N_{\text{BCLK}} &= \left\lfloor \frac{f_{\text{in}}}{f_{\text{BCLK}}} \right\rceil \\ N_{\text{LRCLK}} &= \left\lfloor \frac{f_{\text{in}}}{f_{\text{LRCLK}}} \right\rceil \end{aligned} \tag{12}$$

LRCLK partitions the bit stream into left and right words, toggling once per sample period. However, the MSB is not transmitted immediately on the LRCLK transition. Instead, there is a one BCLK cycle delay before the first bit is sent. This is a defining characteristic of the I²S standard, distinguishing it from similar formats like Left-Justified (where the MSB is sent immediately on the transition) and Right-Justified (where the LSB lands exactly on the next LRCLK edge). After the one-cycle pause, bits are transmitted MSB-first, one per BCLK edge.

2.4. The Voice Mixer

As stated in Chapter 2.3, I²S signals have to satisfy a couple of requirements:

- Its amplitude must be less than or equal to $2^{(B-1)} - 1$
- They must be encoded as a zero-centered signed integer

In order to mix several voices, we need to ensure all those criteria are met. Mathematically, this can be expressed as:

$$y = \left\lfloor \sum_{i=0}^{n} \frac{A_{\max}}{N_{\text{voices(max)}}} \times \text{normalize}(\text{center}(x_i)) \right\rfloor \tag{13}$$

where:

- y is the output signal from the mixer, the one that will go into the I²S module
- x_i is each of the voices
- n is the number of voices available
- $N_{\text{voices(max)}}$ is the maximum amount of voices allowed before clipping
- $A_{\max} = 2^{(B-1)} - 1$ the maximum amplitude that doesn't clip
- $\text{normalize}(x) = x/A(x)$ is a function that outputs the signal x but scaled, such that the amplitude of the signal is 1
- $\text{center}(x)$ makes the signal zero-centered

In the context of this project, where x_i is known to be a square wave of range $[0, 1]$ coupled with an enable signal, with $A(x_i) = \max_t |x_i| = 1$, the following process can be performed:

$$\text{center}(x_i, \text{enable}_i) = \begin{cases} 1 \text{ if enable}_i = 1 \text{ and } x_i = 1 \\ 0 \text{ if enable}_i = 0 \\ -1 \text{ if enable}_i = 1 \text{ and } x_i = 0 \end{cases} \tag{14}$$

$$\begin{aligned} y &= \left\lfloor \sum_{i=0}^{n} \frac{A_{\max}}{N_{\text{voices(max)}}} \times \frac{\text{center}(x_i, \text{enable}_i)}{1} \right\rfloor \\ &= \left\lfloor \frac{2^{(B-1)} - 1}{N_{\text{voices(max)}}} \times \sum_{i=0}^{n} \text{center}(x_i, \text{enable}_i) \right\rfloor \end{aligned} \tag{15}$$

3. Discussion

3.1. Project Hardware & Software

This project is implemented using the MAX 10 FPGA board as the main central processing unit of the system. It handles the digital audio generation and control. For the system to get a sound output, two external components are connected to the MAX 10, which are the MAX98357 amplifier and a 3 W 8 Ω Speaker. The Software that was used to implement the system is Quartus Prime, and the components used VHDL code to code each of the components. In order to satisfy the requirements from Chapter 2, the components Voice Generators, Voice Mixer, I²S Controller, Top Synth, and Synth are implemented using VHDL code. This code uses the behavioral and structural architecture.

Fig. 1. 3 W 8 Ω Speaker

Fig. 2. MAX98357 amplifier

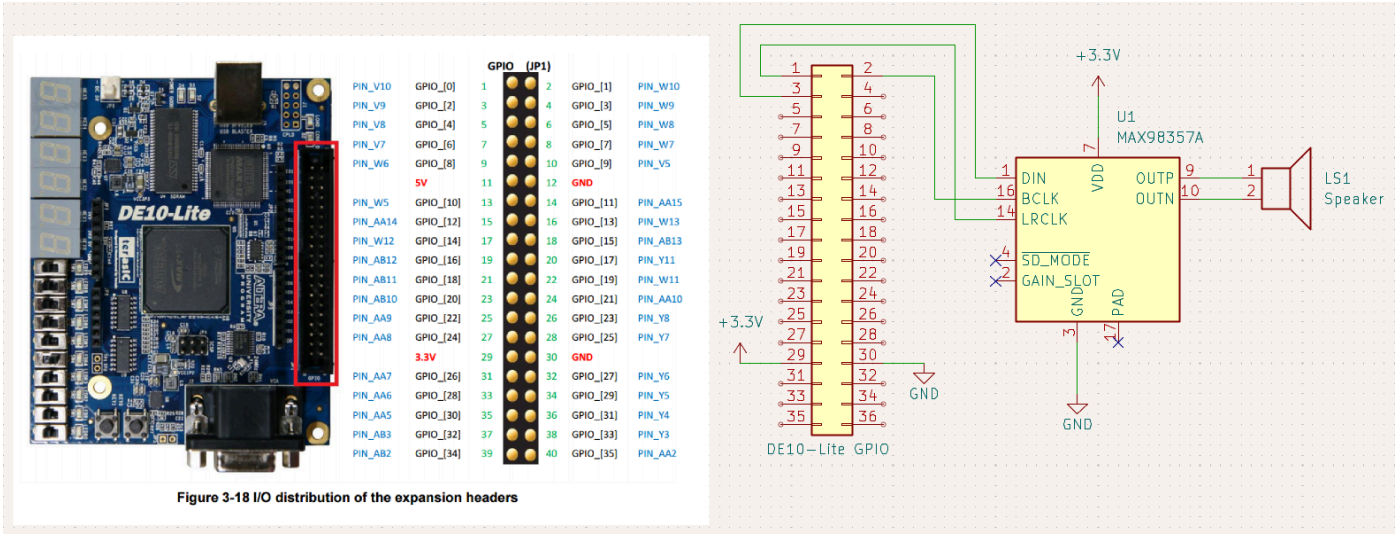

Fig. 3. Electronic schematic

3.2. Voice Generators

This component implements the toggle dividers of Chapter 2.1, which allow the generation of clocks that can output a squarewave signal at specified signals.

- Generic parameters:
  - The number of voices
  - The toggle divider values of N. As commented in the code, those values need to be precomputed. The following Julia script code combines all the math from Chapter 2.1 and Chapter 2.2 to obtain the corresponding values

```julia
# Semitones is an array of distances in semitones from concert A (A4)
f(semitones) = semitones .|> n -> 440*2^(n/12) .|> n -> round(50_000_000/n * 0.5) .|> Int

print([-9, -7, -5, -4, -2, 0, 2, 3])
```

- Inputs:
  - The 50 MHz input clock signal that is used to derive all other signals.
  - A logic vector of active-high enable signals, which signify which voice should be sounding and which one shouldn't.
- Outputs:
  - A logic vector of output signals, where each signal is a square wave between 0 and 1. If the corresponding enable is not HIGH, the output is 0.

3.3. Voice Mixer

The voice mixer in this project performs the logic of Chapter 2.4, centering the square-wave voices and adding them together.

- Generic parameters:
  - The number of voices entering the mixer
  - The number of bits used to represent the output signal
- Inputs:
  - A vector signal containing the value of the square-wave voices
  - A vector signal containing the enabling values
- Outputs:
  - A vector containing the resulting amplitude wave of the mixer

3.4. I²S Controller

The controller implements the logic defined in Chapter 2.3 to convert the amplitude wave into an I²S signal that is sent to an I²S module.

- Generic parameters:
  - The number of bits used for the audio amplitude input signal
- Inputs:
  - The 50 MHz clock signal
  - An active-low reset signal
  - A signal vector containing the input audio amplitude
- Outputs:
  - The bclk and lrclk clock signals

- The data_out serial signal

3.5. Top Synth

It's the top-level component. Using the structural pattern, it connects all of the other components together into a generic component.

- Generic parameters:
  - The number of voices
  - The toggle divider values of N
  - The number of bits used to represent the output signal
- Inputs:
  - The 50 MHz clock signal
  - A vector signal containing the enables values
  - An active-low reset signal
- Outputs:
  - The bclk and lrclk clock signals
  - The data_out serial signal

3.6. Synth

A wrapper for the top synth component because Quartus doesn't like arrays as generics in top-level components.

- Generic parameters:
  - The number of voices
  - The toggle divider values of N
  - The number of bits used to represent the output signal
- Inputs:
  - The 50 MHz clock signal
  - A vector signal containing the enables values
  - An active-low reset signal
- Outputs:
  - The bclk and lrclk clock signals
  - The data_out serial signal

TABLE I
DISTRIBUTION OF WORK FOR DESIGN PROJECT

Team Member	Responsibility
Group	Research
	System Architecture Design
	Block Diagram
	Hardware Testing
	Final Testing
	Project Report
	Project Presentation
	Live Demonstration
Josué	Note Generator Design
	Pin Assignments
	Simulation
	Polyphonic Mixer
Abigail	I²S Controller Design
	Top-Level Module Integration
	Connections

TABLE II
LOGIC DEVICES DESIGN PROJECT TIMELINE

Logic Devices Design Project Timeline								
Month	**March**			**April**				
	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8
Week # / Dates	12-Mar	19-Mar	26-Mar	2-Apr	9-Apr	16-Apr	23-Apr	30-Apr
Task								
Research								
Design System architecture								
Block Diagram								
FPGA Setup								
Note Generator Development								
Polyphony and Mixer								
Hardware Interface								
Debugging and Testing								
Presentation								
Report								
Demonstration								

4. Validation of Data

The idea of breaking the project into several components was that each of them could be tested separately (unit tests), while the two top-level components would be tested by testing the entire system (integration tests).

A first test in all cases is based on the correctness of the code when compared with their respective mathematical models as described in Chapter 3.

A final demo can be found at:

```
1  https://youtu.be/l18Umr7pZi4
```

4.1. Voice Generators

For the voice generators, given that they are coupled to the 50 MHz clock input, they were tested in an oscilloscope. While not every note was tested, using three channels in the oscilloscope, the values for the C3, D3, and E3 notes were tested, as seen in Fig. 4, and compared to the actual values according to the equal temperament table.

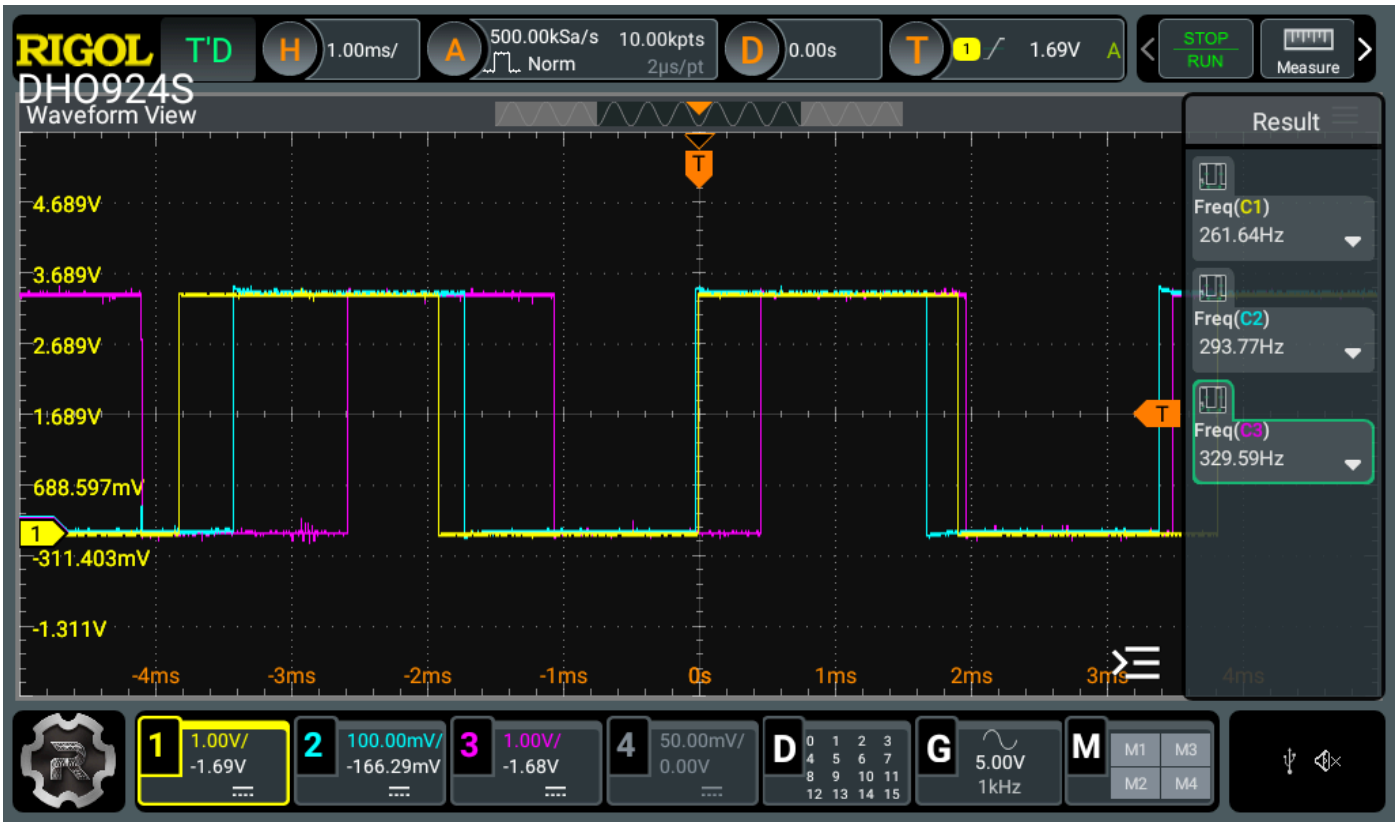

Fig. 4. Oscope readings of the voice generator. Yellow: C3, Light blue: D3, Pink: E3

TABLE III
VOICE GENERATOR MEASUREMENTS

Measured Value (Hz)	Theoretical Value (Hz)	Absolute Difference (Hz)
261.64	261.63	0.01
293.77	293.66	0.11
329.59	329.62	0.03

As it can be seen in Table III, the absolute difference in all cases is less than 0.2 Hz. Furthermore, Fig. 4 shows that all three signals are generated at the same time, proving the ability to perform polyphonic voices.

4.2. Voice Mixer

For the voice mixer, a functional simulation was performed using University Program VWF, as can be seen in Fig. 5.

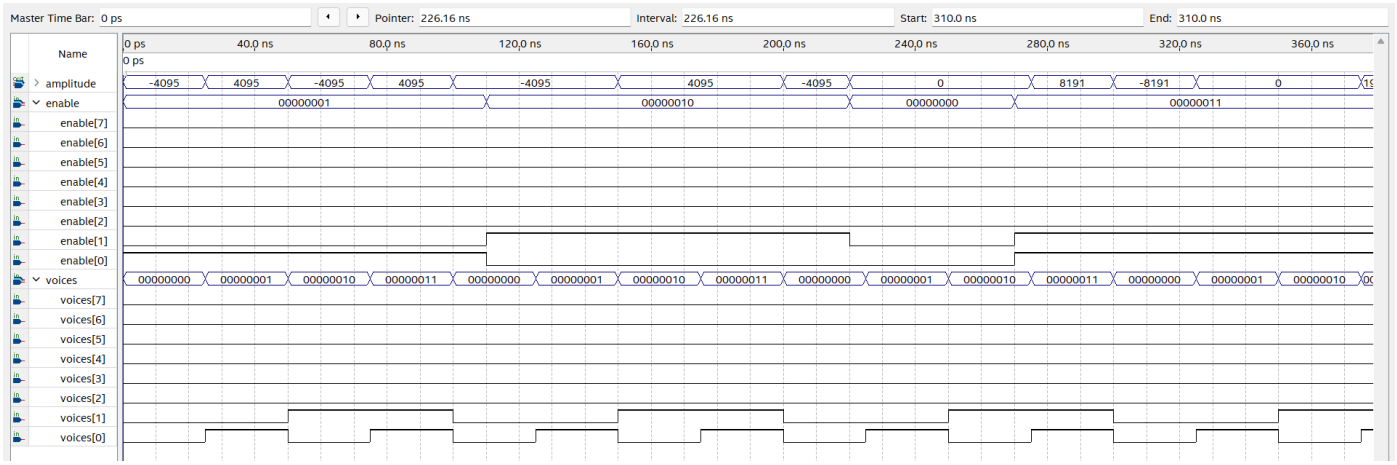

Fig. 5. Simulation of the voice mixer

The simulation intends to show a couple of behaviors that display the good functioning of the mixer:

- During the first period, when `enable[0]` is HIGH, the amplitude oscillates between positive and negative 4095, instead of 0 and 1, showing that it is actually centering the waveform and scaling it correctly. The change of sign of `amplitude` corresponds to the rise and fall of `voices[0]`.
- During the first period, when `enable[1]` is HIGH, the same thing happens, but with `voices[1]`, verifying that independent channels remain independent.
- During the period where both `enable[0]` and `enable[1]` are LOW, the `amplitude` value is 0, showing how the signal is zero-centered (when no voices, value is 0).
- When both `enable[0]` and `enable[1]` are HIGH, the value oscillates from positive and negative 8191 when both `voices[0]` and `voices[1]` coincide (which is approximately double the value of the previous case), and 0 when `voices[0]` and `voices[1]` have opposite values, proving the normalization and centering are working correctly.

4.3. I²S controller

The I²S controller is used to generate the BCLK from the 50 MHz, it also uses the LRCLK to separate the left and right channels and sends 16-bit audio serially and outputs mono audio through the left channel only. For this I²S controller, the expected output was verified through the sound that was heard from the speaker output of the notes C4, D4, E4, F4, G4, A4, B4, C5. The verification was achieved through connecting the voice generator component, the voice mixer, and the I²S controller together in a BDF file. This will make the inputs the switches, the KEY0, and the 50MHz clock on the MAX10, which are the enable, reset, and the clk or clock_50_mhz, respectively, on the bdf components. The outputs from these connected components will be output through the speaker using the bclk, lrclk, and the data_out on the components, which are connected to the amplifiers bclk, lrclk, and din, respectively, from the MAX10 boards' GPIO. So, when the switches were toggled to a HIGH on the board, each sound started playing through the speaker, whether it was individually or together. Another thing that was verified from the i2s controller was that when the reset was pressed at KEY0, no sound was heard through the speaker.

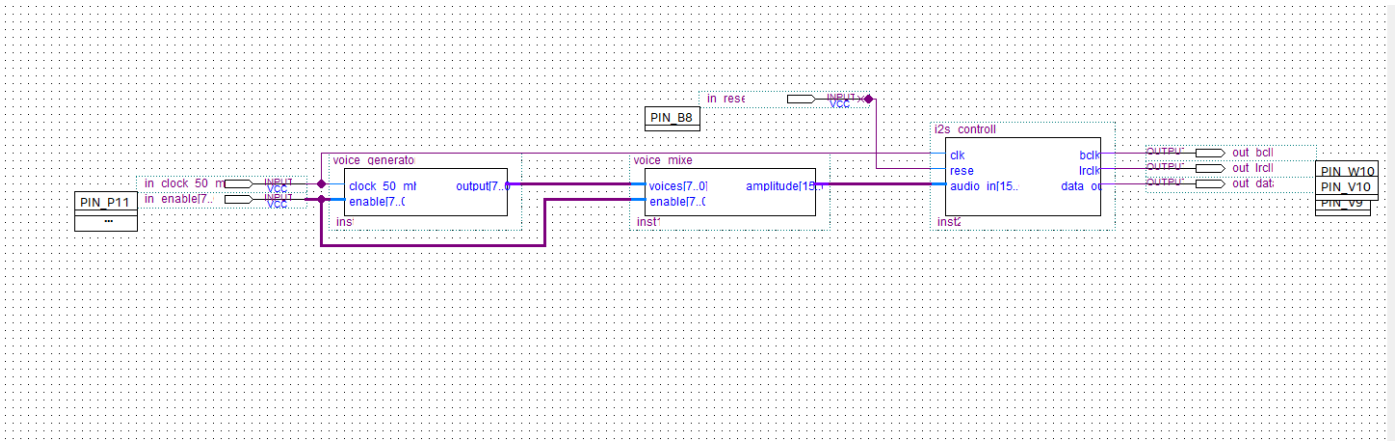

Fig. 6. BDF of the components connected together to make the Synthesizer

5. Code and Schematics

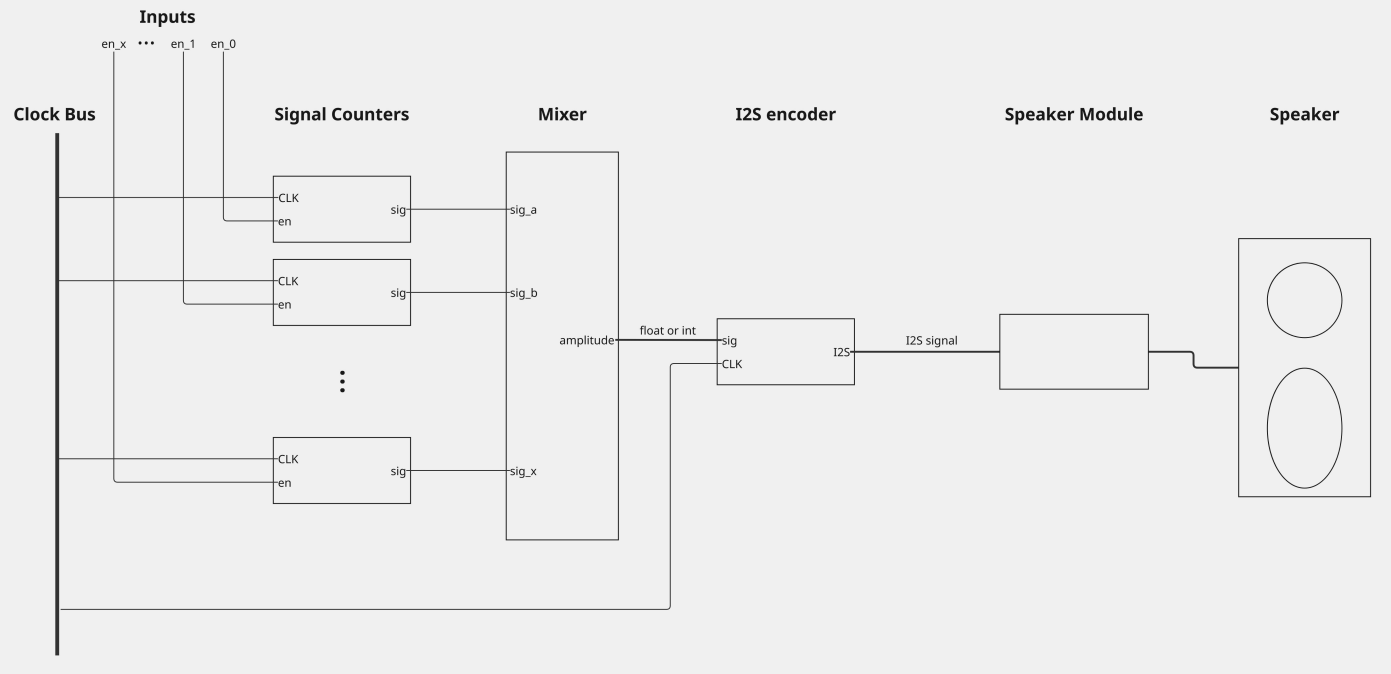

Fig. 7. Full project schematic

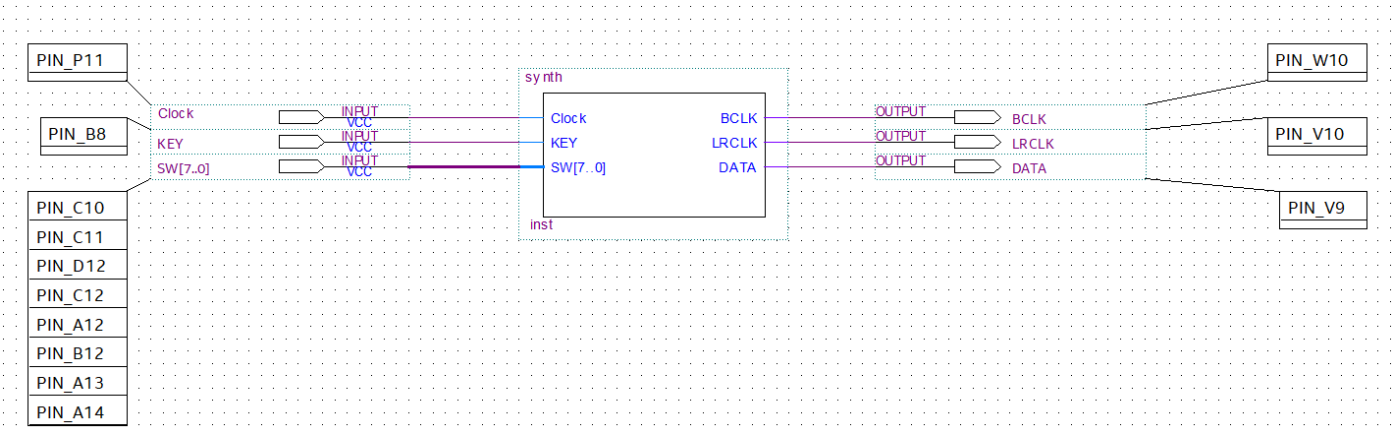

Fig. 8. Block Diagram/Schematic with pin assignments of FPGA top-level component

```vhdl
-- Square wave generator

library ieee;
use ieee.std_logic_1164.all;

entity square_wave_generator is
  generic (
    N: integer := 16
  );
  port (
    clock, enable: in std_logic;
```

```
    output: out std_logic
  );
end square_wave_generator;

architecture hybrid of square_wave_generator is
  signal count_int: integer := 0;
  signal output_buf: std_logic := '0';
begin
  -- Clocking process
  ps: process (clock)
  begin
    if rising_edge(clock) then
      -- When on: generate square wave
      if enable = '1' then
        -- Overflow
        if count_int = N - 1 then
          count_int <= 0;   -- Reset counter
          output_buf <= not output_buf; -- Flip wrap
        -- Normal increment
        else
          count_int <= count_int + 1; -- Increment counter
        end if;
      -- When off: clear
      else
        count_int <= 0;  -- Reset counter
        output_buf <= '0';
      end if;
    end if;
  end process;

  -- Combinatorial assignments
  output <= output_buf;
end architecture;


-- Voice generator

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
use work.synth_pkg.all;

```

```vhdl

entity voice_generators is
  generic (
    NUM_VOICES: natural := 8;
    FREQUENCY_DIVIDER_COUNTS: freq_count_array_t := (47778, 50619, 56818, 63776, 71587,
    75843, 85131, 95551)
  );
  port (
    clock_50_mhz: in std_logic;
    enable: in std_logic_vector(NUM_VOICES-1 downto 0);
    output: out std_logic_vector(NUM_VOICES-1 downto 0)
  );
end voice_generators;

architecture hybrid of voice_generators is
  signal output_buffer: std_logic_vector(FREQUENCY_DIVIDER_COUNTS'length-1 downto 0);

begin

  -- Ensure NUM_VOICES = FREQUENCY_DIVIDER_COUNTS'length
  assert FREQUENCY_DIVIDER_COUNTS'length = NUM_VOICES
    report "NUM_VOICES must match FREQUENCY_DIVIDER_COUNTS length"
    severity failure;

  pulse_gen: for i in 0 to NUM_VOICES-1 generate
    pulse: entity work.square_wave_generator
      generic map (
        -- N = 2 * round(50MHz / f_out)  <- Precalculate
        N => FREQUENCY_DIVIDER_COUNTS(i)
      )
      port map (
        clock => clock_50_mhz,
        enable => enable(i),
        output => output_buffer(i)
      );
  end generate;

  output <= output_buffer;

end architecture;
```

Listing 1: `voice_generators.vhd`

```vhdl
library ieee;
use ieee.std_logic_1164.all;
```

```
3   use ieee.numeric_std.all;
4
5   entity voice_mixer is
6     generic (
7       IN_VOICES_NUMBER: natural := 8;
8       MAX_VOICES: natural := 8;
9       OUT_BITS_NUMBER: natural := 16
10    );
11     port(
12      voices: in std_logic_vector(IN_VOICES_NUMBER-1 downto 0);
13        enable: in std_logic_vector(IN_VOICES_NUMBER-1 downto 0);
14        amplitude: out std_logic_vector(OUT_BITS_NUMBER-1 downto 0)
15    );
16  end voice_mixer;
17
18  architecture behavioral of voice_mixer is
19  begin
20
21      ps: process(enable, voices)
22          variable count_integer: integer;
23      begin
24          -- Initialize variables
25        count_integer := 0;
26
27          -- Grab each voice signal (0 to 1 square wave),
28          -- interpolate it into a -1 to 1 square wave,
29          -- and add it to the counter
30          -- Disabled waves have value 0
31          for i in 0 to IN_VOICES_NUMBER - 1 loop
32              if enable(i) = '1' then
33                  if voices(i) = '1' then
34                      count_integer := count_integer + 1;
35                  else
36                      count_integer := count_integer - 1;
37                  end if;
38              end if;
39          end loop;
40
41          -- Performs scaling and conversion to signed integer
42          amplitude <= std_logic_vector(
43        to_signed(
44          count_integer * (2**(OUT_BITS_NUMBER - 1) - 1)/MAX_VOICES,
45          OUT_BITS_NUMBER
46        )
```

```
47       );
48     end process;
49 end architecture;
```

Listing 2: `voice_mixer.vhd`

```VHDL
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity i2s_controller is
6    generic (
7      AUDIO_IN_BITS: natural := 16
8    );
9    port (
10     clk  : in std_logic; --50MHz
11     reset : in std_logic;
12     audio_in : in std_logic_vector( AUDIO_IN_BITS-1 downto 0); --mono sample
13     bclk, lrclk, data_out: out std_logic
14
15   );
16 end i2s_controller;
17
18 architecture behavior of i2s_controller is
19
20 signal bclk_reg    : std_logic:='0';
21 signal lrclk_reg     : std_logic :='0';
22
23 signal clk_div : integer := 0;
24 signal bit_cnt : integer range 0 to 2*AUDIO_IN_BITS - 1 := 0;
25
26 signal shift_reg : std_logic_vector (AUDIO_IN_BITS-1 downto 0 );
27
28 begin
29
30 --Generate the BCLK ( 1.6 MHz from 50 MHz)
31 --50MHz /(2x16) = 1.6 MHz
32 bclk_gen: process(clk)
33 begin
34     if rising_edge(clk) then
35         clk_div <= clk_div +1;
36
37         if clk_div = AUDIO_IN_BITS then
38             clk_div <= 0;
39             bclk_reg <= not bclk_reg; --BCLK toggles
```

```
40          end if;
41      end if;
42  end process;
43
44  bclk <=bclk_reg;
45
46  -- Shift data on BCLK
47
48  shift: process(bclk_reg, reset)
49  begin
50      if reset ='0' then
51          bit_cnt<= 0 ;
52          lrclk_reg<='0';
53      elsif rising_edge(bclk_reg) then
54          if bit_cnt = 0 then
55               shift_reg <= audio_in;
56               lrclk_reg <= '0';  -- Left channel
57          elsif bit_cnt = AUDIO_IN_BITS then
58               shift_reg <= (others => '0');  -- Right channel (silent for mono)
59               lrclk_reg <= '1';
60          else
61               shift_reg <= shift_reg(AUDIO_IN_BITS-2 downto 0) & '0';
62          end if;
63
64          data_out <= shift_reg(AUDIO_IN_BITS-1);
65          bit_cnt <= bit_cnt + 1;
66
67          if bit_cnt = 2*AUDIO_IN_BITS-1 then
68               bit_cnt <= 0;
69          end if;
70      end if;
71  end process;
72
73  lrclk <= lrclk_reg;
74
75  end behavior;
```

Listing 3: `i2s_controller.vhd`

```
1  -- synth_pkg.vhd                                                    VHDL
2  package synth_pkg is
3      type freq_count_array_t is array (natural range <>) of natural;
4  end package;
```

Listing 4: `synth_pkg.vhd`

```vhdl
--top level entity
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library work;
use work.synth_pkg.all;

entity top_synth is
  generic (
    NUM_VOICES: natural := 8;
    -- C4, D4, E4, F4, G4, A4, B4, C5
    -- Distance in semitones from concert A (A4) - distance to frequency - frequency to count - Make it an integer
    -- [-9, -7, -5, -4, -2, 0, 2, 3] .|> n -> 440*2^(n/12) .|> n -> round(50_000_000/n * 0.5) .|> Int
    FREQUENCY_DIVIDER_COUNTS: freq_count_array_t := (47778, 50619, 56818, 63776, 71587, 75843, 85131, 95551);
    MAX_VOICES: natural := 8;

    OUT_BITS_NUMBER: natural := 16
  );
  port(
    -- INPUTS
    Clock, KEY: in std_logic; -- clock and reset
    SW: in std_logic_vector(NUM_VOICES - 1 downto 0); --switches

    -- OUTPUTS
    BCLK, LRCLK, DATA : out std_logic
  );
end top_synth;

architecture structural of top_synth is

-- internal signal
  signal voices_sig: std_logic_vector (NUM_VOICES - 1 downto 0);
  signal amplitude_sig: std_logic_vector (OUT_BITS_NUMBER - 1 downto 0);
begin

  --voice generator component connections
  voice_gen : entity work.voice_generators
    generic map (
      NUM_VOICES => NUM_VOICES,
      -- C4, D4, E4, F4, G4, A4, B4, C5
```

```
42        -- Distance in semitones from concert A (A4) - distance to frequency - frequency to
          count - Make it an integer
43        -- [-9, -7, -5, -4, -2, 0, 2, 3] .|> n -> 440*2^(n/12) .|> n -> round(50_000_000/n *
          0.5) .|> Int
44        FREQUENCY_DIVIDER_COUNTS => (47778, 50619, 56818, 63776, 71587, 75843, 85131, 95551)
45      )
46      port map(
47        clock_50_mhz => Clock,
48        enable => SW,
49        output => voices_sig
50      );
51    --mixer component connections
52    mixer: entity work.voice_mixer
53      generic map (
54        IN_VOICES_NUMBER => NUM_VOICES,
55        MAX_VOICES => MAX_VOICES,
56        OUT_BITS_NUMBER => OUT_BITS_NUMBER
57      )
58      port map (
59        voices=> voices_sig,
60        enable=> SW,
61        amplitude=> amplitude_sig
62      );
63    --i2s component connections
64    i2s: entity work.i2s_controller
65      generic map (
66        AUDIO_IN_BITS => OUT_BITS_NUMBER
67      )
68      port map (
69        clk  => Clock,
70           reset => KEY,
71           audio_in => amplitude_sig,
72           bclk=> BCLK,
73        lrclk=> LRCLK,
74        data_out=> DATA
75      );
76  end architecture;
```

Listing 5: `top_synth.vhd`

```VHDL
1   library ieee;
2   use ieee.std_logic_1164.all;
3
4   entity synth is
5     port(
```

```
6       -- INPUTS
7       Clock, KEY: in std_logic; -- clock and reset
8       SW: in std_logic_vector(8-1 downto 0); --switches
9
10      -- OUTPUTS
11      BCLK, LRCLK, DATA : out std_logic
12    );
13  end synth;
14
15  architecture structural of synth is
16  begin
17    s: entity work.top_synth
18      generic map (
19        NUM_VOICES => 8,
20        -- C4, D4, E4, F4, G4, A4, B4, C5
21        -- Distance in semitones from concert A (A4) - distance to frequency - frequency to
          count - Make it an integer
22        -- [-9, -7, -5, -4, -2, 0, 2, 3] .|> n -> 440*2^(n/12) .|> n -> round(50_000_000/n *
          0.5) .|> Int
23        FREQUENCY_DIVIDER_COUNTS => (47778, 50619, 56818, 63776, 71587, 75843, 85131, 95551),
24        MAX_VOICES => 8,
25
26        OUT_BITS_NUMBER => 16
27      )
28      port map (
29        Clock, KEY, SW,
30        BCLK, LRCLK, DATA
31      );
32
33  end architecture;
```

Listing 6: `synth.vhd`

6. Conclusion

The project successfully demonstrated the design and implementation of a polyphonic square wave synthesizer using an FPGA-based architecture, utilizing the behavioral and structural styles. The mathematical foundations of generating sound, like frequency division, equal temperament tuning, and digital signal constraints for I²S audio, were effectively translated into hardware through modular VHDL components. Each of the subsystems, which are the voice generators, mixer, and I²S controller, was independently verified and then integrated into a complete system that could produce clear and accurate audio output. This project concretely illustrated the central advantage of FPGA-based audio synthesis: all 8 oscillators ran as independent, simultaneous hardware datapaths with no scheduling overhead, a degree of true parallelism that would be impractical to replicate in software at the same sample rate.